whole content of the file is buffered in memory and on disk before you receive it. Then you can use the `OpenReadStream` method to read the data out.

If users post large files to your website, you may start to run out of space in memory or on disk as ASP.NET Core buffers each of the files. In that case, you may need to stream the files directly to avoid saving all the data at the same time. Unfortunately, unlike the model-binding approach, streaming large files can be complex and error-prone, so it's outside the scope of this book. For details, see Microsoft's documentation at http://mng.bz/MBgn.

TIP Don't use the `IFormFile` interface to handle large file uploads, as you may see performance problem. Be aware that you can't rely on users *not* to upload large files, so avoid file uploads when you can!

For the vast majority of minimal API endpoints, the default configuration of model binding for simple and complex types works perfectly well. But you may find some situations in which you need to take a bit more control.

7.7 Custom binding with BindAsync

The model binding you get out of the box with minimal APIs covers most of the common situations that you'll run into when building HTTP APIs, but there are always a few edge cases in which you can't use it.

You've already seen that you can inject `HttpContext` into your endpoint handlers, so you have direct access to the request details in your handler, but often, you still want to encapsulate the logic for extracting the data you need. You can get the best of both worlds in minimal APIs by implementing `BindAsync` in your endpoint handler parameter types and